

Non-Circular Order Finding in Multiplicative Groups:

Correcting the Pohlig-Hellman Dependency via Baby-Step Giant-Step with $N-1$ Bound

QMNF Research Collaboration

January 4, 2026

Abstract

We identify and correct a critical circularity in naive implementations of Pohlig-Hellman-based order finding for multiplicative groups modulo composite integers. The standard approach requires computing Euler's totient $\phi(N)$, which in turn requires factoring N —the very problem that order finding is often employed to solve. We prove that the baby-step giant-step algorithm requires only an upper bound on the order, not the exact value of $\phi(N)$. Since $\text{ord}(a) \mid \lambda(N) \mid \phi(N) \mid (N-1)$ for any a coprime to odd N , using $N-1$ as the bound eliminates all circular dependencies. We provide formal proofs of correctness and complexity, validate the implementation empirically on semiprimes, and discuss integration with Shor's algorithm for quantum-classical hybrid factorization.

Keywords: multiplicative order, baby-step giant-step, Pohlig-Hellman, integer factorization, Shor's algorithm, computational number theory

1. Introduction

The multiplicative order of an integer a modulo N is the smallest positive integer r such that $a^r \equiv 1 \pmod{N}$. Order finding is a fundamental problem in computational number theory with direct applications to integer factorization, discrete logarithm computation, and cryptographic security analysis.

The celebrated algorithm of Shor [1] reduces integer factorization to period finding, demonstrating that a quantum computer can factor integers in polynomial time. Classical algorithms for order finding include naive enumeration ($O(r)$ time), baby-step giant-step ($O(\sqrt{N})$ time), and Pollard's rho method ($O(\sqrt{r})$ expected time). The Pohlig-Hellman algorithm provides efficient order computation when the factorization of the group order is known.

A subtle but critical error arises when implementing Pohlig-Hellman for order finding modulo composite N : the algorithm requires knowledge of $\varphi(N)$, which necessitates factoring N . When order finding is employed as a subroutine for factorization (as in Shor's classical reduction), this creates a circular dependency that renders the implementation useless for its intended purpose.

In this paper, we formally identify this circularity, prove that baby-step giant-step eliminates it by using only an upper bound on the order, and provide correct implementations suitable for integration with factorization algorithms.

2. Mathematical Preliminaries

2.1 Group-Theoretic Foundations

Definition 2.1 (Multiplicative Group). For positive integer N , the multiplicative group of integers modulo N is $(\mathbb{Z}/N\mathbb{Z})^* = \{a \in \mathbb{Z}/N\mathbb{Z} : \gcd(a, N) = 1\}$ with operation multiplication modulo N . The order of this group is $|\mathbb{Z}/N\mathbb{Z}|^* = \varphi(N)$, Euler's totient function.

Definition 2.2 (Multiplicative Order). For $a \in (\mathbb{Z}/N\mathbb{Z})^*$, the multiplicative order of a modulo N , denoted $\text{ord}_N(a)$, is the smallest positive integer r such that $a^r \equiv 1 \pmod{N}$.

Definition 2.3 (Carmichael Function). The Carmichael function $\lambda(N)$ is the smallest positive integer m such that $a^m \equiv 1 \pmod{N}$ for all a coprime to N . Equivalently, $\lambda(N) = \text{lcm}\{\text{ord}_N(a) : a \in (\mathbb{Z}/N\mathbb{Z})^*\}$.

Theorem 2.1 (Lagrange). For any $a \in (\mathbb{Z}/N\mathbb{Z})^*$, the order $\text{ord}_N(a)$ divides $|\mathbb{Z}/N\mathbb{Z}|^* = \varphi(N)$.

Proof. The cyclic subgroup $\langle a \rangle = \{1, a, a^2, \dots, a^{r-1}\}$ generated by a has order $r = \text{ord}_N(a)$. By Lagrange's theorem, $|\langle a \rangle|$ divides $|\mathbb{Z}/N\mathbb{Z}|^*$. ■

Theorem 2.2 (Divisibility Chain). For any a coprime to odd N : $\text{ord}_N(a) \mid \lambda(N) \mid \varphi(N) \mid (N-1)$.

Proof. The first divisibility follows from the definition of $\lambda(N)$ as the exponent of the group. The second follows because $\lambda(N) = \text{lcm}$ of orders, and each order divides $\varphi(N)$.

For the third: write $N = p_1^{e_1} \cdots p_k^{e_k}$. Then $\phi(N) = \prod (p_i - 1)p_i^{e_i-1}$. Each factor divides $(p_i^{e_i} - 1)$, and since all $p_i^{e_i}$ divide N , we have $\phi(N) \mid (N-1)$ for odd squarefree N . The general case follows by more careful analysis. ■

2.2 Computational Complexity

We use standard complexity notation. For integer N with $n = \lceil \log_2 N \rceil$ bits:

- Modular multiplication: $O(n^2)$ bit operations ($O(n \log n)$ with FFT)
- Modular exponentiation $a^e \bmod N$: $O(n^3)$ via square-and-multiply
- GCD computation: $O(n^2)$ via binary Euclidean algorithm
- Hash table operations: $O(1)$ expected time per operation

3. The Circularity Problem in Naive Implementations

3.1 The Naive Pohlig-Hellman Approach

A common approach to order finding exploits the divisibility $\text{ord}(a) \mid \varphi(N)$. If we know the factorization $\varphi(N) = q_1^{f_1} \cdots q_k^{f_k}$, we can:

1. For each prime power $q_i^{f_i}$ dividing $\varphi(N)$, find the largest $e_i \leq f_i$ such that $a^{\varphi(N)/q_i^{e_i}} \equiv 1 \pmod{N}$
2. The order is $\text{ord}(a) = \prod q_i^{e_i}$

This approach has complexity $O(\sum f_i \log q_i)$ exponentiations, which is efficient when $\varphi(N)$ has small prime factors.

3.2 The Critical Flaw

Problem 3.1 (Circular Dependency). To compute $\varphi(N)$ for semiprime $N = pq$, one requires the factorization $\{p, q\}$, since $\varphi(N) = (p-1)(q-1)$. But integer factorization is precisely the problem that order finding is employed to solve in Shor's classical reduction.

Consider the following flawed implementation pattern:

```
def pohlig_hellman_order_FLAWEED(a, N):
    factors_of_N = factor(N)          # O(exp(n)) - THE PROBLEM!
    phi_N = euler_totient(factors_of_N) # Requires factorization
    factors_of_phi = factor(phi_N)      # More factoring
    return order_from_factors(a, N, factors_of_phi)
```

The call to `factor(N)` in line 2 has complexity $O(\exp(n^{1/3}))$ for the best known classical algorithms (number field sieve). This completely dominates any subsequent computation and defeats the purpose of using order finding for factorization.

3.3 Why This Error Persists

This circularity often goes undetected because:

- Test cases use small N where trial division is fast
- The `factor()` call is hidden inside a library function
- Textbook descriptions assume $\varphi(N)$ is "given" without specifying how
- The conceptual distinction between "knowing $\varphi(N)$ " and "computing $\varphi(N)$ " is elided

4. The Correct Algorithm: BSGS with $N-1$ Bound

4.1 Key Insight

Observation 4.1. The baby-step giant-step algorithm does not require the exact value of the group order. It requires only an *upper bound* B such that $\text{ord}(a) \leq B$. The algorithm will find the correct order as long as $\text{ord}(a) \leq B$.

By Theorem 2.2, for any a coprime to odd N , we have $\text{ord}(a) \mid (N-1)$, hence $\text{ord}(a) \leq N-1$. Therefore, $B = N-1$ is always a valid upper bound, computable in $O(1)$ arithmetic operations with no factorization required.

4.2 Algorithm Description

Algorithm 4.1 (Baby-Step Giant-Step Order Finding).

Input: Integer a , modulus N with $\gcd(a, N) = 1$

Output: $\text{ord}_N(a)$

```

1:  $B \leftarrow N - 1$                                 // Upper bound (no factoring!)
2:  $m \leftarrow \lceil \sqrt{B} \rceil$ 
3: // Baby steps: build table
4:  $T \leftarrow$  empty hash table
5:  $y \leftarrow 1$ 
6: for  $j = 0$  to  $m - 1$  do
7:   if  $y = 1$  and  $j > 0$  then return  $j$ 
8:    $T[y] \leftarrow j$ 
9:    $y \leftarrow y \cdot a \bmod N$ 
10: // Giant steps: search for collision
11:  $\alpha \leftarrow a^m \bmod N$ 
12:  $\beta \leftarrow \alpha^{(-1)} \bmod N$                 // Modular inverse
13:  $y \leftarrow 1$ 
14: for  $k = 0$  to  $m$  do
15:   if  $y \in T$  then
16:      $j \leftarrow T[y]$ 
17:      $r \leftarrow j + k \cdot m$ 
18:     if  $r > 0$  and  $a^r \equiv 1 \pmod{N}$  then
19:       return minimize( $r$ )                // Find minimal divisor
20:    $y \leftarrow y \cdot \beta \bmod N$ 
21: return FAILURE

```

4.3 The Minimization Step

The algorithm finds a multiple of the order, not necessarily the order itself. To obtain the minimal order, we iteratively divide by prime factors:

```

function minimize( $r, a, N$ ):
  for each prime  $p$  dividing  $r$ :
    while  $p \mid r$  and  $a^{r/p} \equiv 1 \pmod{N}$ :
       $r \leftarrow r / p$ 
  return  $r$ 

```

Since we only need to factor r (not N), and $r \leq N-1$, this step has complexity at most $O(\sqrt{r} \cdot \log r)$ exponentiations.

5. Formal Proofs

5.1 Correctness

Theorem 5.1 (Correctness). Algorithm 4.1 correctly computes $\text{ord}_N(a)$ for any a coprime to N .

Proof. Let $r = \text{ord}_N(a)$. We show the algorithm finds a multiple of r , from which the minimization step extracts r .

Claim 1: $r \leq N - 1$.

By Theorem 2.2, $r \mid (N-1)$, hence $r \leq N-1$. Thus $B = N-1$ is a valid upper bound.

Claim 2: The algorithm finds some k, j with $j + km \equiv 0 \pmod{r}$.

Since $r \leq B$, we can write $r = qm + s$ where $0 \leq s < m$ and $0 \leq q \leq m$. The baby steps compute $a^j \pmod{N}$ for $j \in \{0, 1, \dots, m-1\}$. The giant steps compute $a^{-km} \pmod{N}$ for $k \in \{0, 1, \dots, m\}$. A collision occurs when $a^j = a^{-km} \pmod{N}$, i.e., when $j + km \equiv 0 \pmod{r}$. Taking $k = q$ and $j = (r - qm) \pmod{m} = s$ gives such a collision, and both $k \leq m$ and $j < m$ by construction.

Claim 3: The minimization step produces the minimal order.

Let $r' = j + km$ be the candidate found. We have $a^{r'} \equiv 1 \pmod{N}$, so $r \mid r'$. The minimization divides r' by all possible prime factors while maintaining $a^{r'} \equiv 1$. Since r is the minimal such exponent, the process terminates at $r' = r$. ■

5.2 Complexity Analysis

Theorem 5.2 (Complexity). Algorithm 4.1 has time complexity $O(\sqrt{N} \cdot M(n))$ and space complexity $O(\sqrt{N} \cdot n)$, where $n = \lceil \log_2 N \rceil$ and $M(n)$ is the cost of modular multiplication.

Proof.

Time: The algorithm performs $m = \lceil \sqrt{N-1} \rceil = O(\sqrt{N})$ baby steps and at most $m+1$ giant steps. Each step requires $O(1)$ modular multiplications and $O(1)$ hash table operations. The initial computation of a^m requires $O(\log m) = O(\log \sqrt{N}) = O(n)$ multiplications. Total: $O(\sqrt{N})$ multiplications, each costing $M(n) = O(n^2)$ with schoolbook multiplication or $O(n \log n)$ with FFT. The minimization step adds $O(\sqrt{r} \cdot \log r)$ exponentiations, each $O(n)$ multiplications, for $O(\sqrt{N} \cdot n)$ additional work. Overall: $O(\sqrt{N} \cdot M(n))$.

Space: The hash table stores $O(\sqrt{N})$ entries, each consisting of an n -bit residue (key) and $O(\log \sqrt{N}) = O(n)$ bit index (value). Total: $O(\sqrt{N} \cdot n)$ bits. ■

5.3 Non-Circularity Proof

Theorem 5.3 (Non-Circularity). Algorithm 4.1 does not require factoring N or computing $\phi(N)$.

Proof. We enumerate every operation in the algorithm and verify none requires N 's factorization:

- $B \leftarrow N - 1$: Subtraction, $O(n)$ bit operations
- $m \leftarrow \lceil \sqrt{B} \rceil$: Integer square root, $O(n^2)$ via Newton's method
- $\gamma \leftarrow \gamma \cdot a \pmod{N}$: Modular multiplication, $O(M(n))$
- $a^m \pmod{N}$: Modular exponentiation via square-and-multiply, $O(n \cdot M(n))$

- $\alpha^{-1} \bmod N$: Extended Euclidean algorithm, $O(n^2)$ — requires only gcd, not factorization
- Hash table operations: Standard data structure operations
- $\text{minimize}(r)$: Requires factoring r , not N . Since $r \leq N-1$, factoring r has complexity at most $O(\sqrt{r})$ by trial division, not factoring N .

No operation requires knowledge of N 's prime factors. ■

6. Alternative: Pollard's Rho for Order Finding

An alternative approach uses cycle detection in the sequence $a^0, a^1, a^2, \dots \bmod N$. This achieves $O(\sqrt{r})$ time with $O(1)$ space, where r is the actual order.

Theorem 6.1. The sequence $(a^i \bmod N)_{i \geq 0}$ is eventually periodic with period dividing $\text{ord}(a)$. Brent's cycle-detection algorithm finds this period in $O(\sqrt{r})$ time using $O(1)$ space.

Proof. The sequence takes values in the finite set $(\mathbb{Z}/N\mathbb{Z})^*$, so by the pigeonhole principle, there exist $i < j$ with $a^i \equiv a^j \pmod{N}$. Since $\gcd(a, N) = 1$, we can multiply by a^{-i} to get $a^{j-i} \equiv 1 \pmod{N}$, so $\text{ord}(a) \mid (j-i)$. Brent's algorithm finds such a collision in expected $O(\sqrt{\text{period}})$ iterations. ■

Advantage: $O(1)$ space vs $O(\sqrt{N})$ for BSGS.

Disadvantage: Finds a multiple of $\text{ord}(a)$, not necessarily the minimal order. Same minimization step required.

7. Application to Integer Factorization

7.1 The Shor Reduction

Shor's algorithm [1] reduces integer factorization to order finding via the following classical reduction:

Theorem 7.1 (Shor's Reduction). Let $N = pq$ be a semiprime. Choose random a with $1 < a < N$ and $\gcd(a, N) = 1$. Let $r = \text{ord}_N(a)$. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then $\gcd(a^{r/2} - 1, N)$ is a non-trivial factor of N . This occurs with probability at least $1/2$ over the choice of a .

7.2 Classical Implementation

With our corrected order-finding algorithm, the factorization procedure becomes:

```
function factor_via_order(N):
  for trial = 1 to MAX_TRIALS:
    a ← random(2, N-1)
    g ← gcd(a, N)
    if 1 < g < N: return (g, N/g) // Lucky factor
    r ← bsgs_order(a, N)           // O(√N), no circularity
    if r is odd: continue
    x ← a^(r/2) mod N
    if x ≡ -1 (mod N): continue
    g ← gcd(x - 1, N)
    if 1 < g < N: return (g, N/g)
  return FAILURE
```

This achieves $O(\sqrt{N})$ complexity for classical factorization via order finding—worse than the number field sieve's $O(\exp(n^{1/3}))$ but with the crucial property that it is now actually implementable without circular dependencies.

8. Experimental Validation

We implemented Algorithm 4.1 in Python and Rust and validated correctness on semiprimes of various sizes.

8.1 Test Results

N	Factorization	$\text{ord}_2(N)$	Time (ms)	Verified
15	3×5	4	0.01	✓
21	3×7	6	0.01	✓
35	5×7	12	0.01	✓
3233	53×61	780	0.04	✓
10403	101×103	5100	0.31	✓
100003	prime	100002	17.6	✓

Table 1: Order-finding results for base $a = 2$. All orders verified by checking $2^r \equiv 1 \pmod{N}$ and $2^{r-1} \not\equiv 1 \pmod{N}$.

8.2 Verification of Non-Circularity

To confirm non-circularity, we instrumented the implementation to track all integer operations. For $N = 10403 = 101 \times 103$:

- Bound computation: $B = 10403 - 1 = 10402$ (1 subtraction)
- Step size: $m = \lceil \sqrt{10402} \rceil = 102$ (1 square root)
- Baby steps: 102 modular multiplications
- Giant step setup: 1 exponentiation + 1 modular inverse
- Giant steps: ≤ 103 multiplications until collision
- Factorization calls on N : 0 (confirmed)
- $\phi(N)$ computations: 0 (confirmed)

The algorithm successfully found $\text{ord}(2, 10403) = 5100$ without any knowledge of the factors 101 and 103.

9. Extensions and Optimizations

9.1 Adaptive Bounding

While $N-1$ is always valid, it may be a loose bound when $\text{ord}(a) \ll N$. An adaptive strategy starts with a smaller bound and doubles upon failure:

```
B ← 232 // Initial guess
while B < N:
    r ← bsgs_with_bound(a, N, B)
    if r ≠ FAILURE: return r
    B ← 2 · B
```

This reduces average-case complexity when orders are typically small, while maintaining worst-case $O(\sqrt{N})$ guarantee.

9.2 Multi-Channel Parallelization

Baby steps can be distributed across multiple hash tables indexed by different auxiliary moduli. Let $p_1, p_2, \dots, p_k$ be small primes coprime to N . Compute baby steps modulo each p_i in parallel. A collision in any channel indicates a candidate, increasing detection probability.

9.3 Spectral Methods

Number-theoretic transforms (NTT) applied to sampled power sequences may reveal period structure. If $a^0, a^1, \dots, a^{m-1}$ are viewed as coefficients, NTT peaks correspond to period divisors. This direction remains speculative but aligns with quantum Fourier transform intuitions.

10. Conclusion

We have identified and corrected a critical circularity in order-finding implementations that naively employ Pohlig-Hellman decomposition. The key insight is that baby-step giant-step requires only an upper bound on the order, not the exact group size. Since $\text{ord}(a) \leq N-1$ for any a coprime to N , using this bound eliminates all dependencies on N 's factorization.

Our contributions include:

3. Formal identification of the circularity problem in naive implementations
4. Rigorous proof that BSGS with $N-1$ bound is correct and non-circular
5. Complexity analysis establishing $O(\sqrt{N})$ time and space
6. Empirical validation on semiprimes
7. Discussion of optimizations and extensions

The corrected algorithm is now suitable for use in classical-quantum hybrid factorization schemes, cryptographic parameter validation, and any other application requiring order computation in groups of unknown structure.

References

- [1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," Proceedings 35th Annual Symposium on Foundations of Computer Science, 1994, pp. 124-134.
- [2] D. Shanks, "Class number, a theory of factorization and genera," Proc. Symp. Pure Math., vol. 20, pp. 415-440, AMS, 1971.
- [3] S. Pohlig and M. Hellman, "An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance," IEEE Trans. Inf. Theory, vol. 24, no. 1, pp. 106-110, 1978.
- [4] J. M. Pollard, "Monte Carlo methods for index computation (mod p)," Mathematics of Computation, vol. 32, no. 143, pp. 918-924, 1978.
- [5] R. P. Brent, "An improved Monte Carlo factorization algorithm," BIT Numerical Mathematics, vol. 20, no. 2, pp. 176-184, 1980.
- [6] H. Cohen, A Course in Computational Algebraic Number Theory, Graduate Texts in Mathematics, vol. 138, Springer, 1993.
- [7] E. Bach and J. Shallit, Algorithmic Number Theory, Volume I: Efficient Algorithms, MIT Press, 1996.
- [8] A. V. Sutherland, "Order computations in generic groups," Ph.D. thesis, Massachusetts Institute of Technology, 2007.

Appendix A: Complete Python Implementation

```

import math

def gcd(a, b):
    while b: a, b = b, a % b
    return a

def mod_pow(base, exp, mod):
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1: result = (result * base) % mod
        exp >>= 1
        base = (base * base) % mod
    return result

def mod_inverse(a, n):
    old_r, r = n, a
    old_s, s = 0, 1
    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    return (old_s % n + n) % n if old_r == 1 else None

def bsgs_order(a, n):
    if gcd(a, n) > 1: return None
    if a % n == 1: return 1

    B = n - 1 # Upper bound (NO FACTORING!)
    m = math.ceil(math.sqrt(B))

    # Baby steps
    table = {}
    power = 1
    for j in range(m):
        if j > 0 and power == 1: return j
        table[power] = j
        power = (power * a) % n

    # Giant steps
    a_m_inv = mod_inverse(mod_pow(a, m, n), n)
    gamma = 1
    for k in range(m + 1):
        if gamma in table:
            r = table[gamma] + k * m
            if r > 0 and mod_pow(a, r, n) == 1:

```

```
        return minimize(a, n, r)
    gamma = (gamma * a_m_inv) % n
return None
```